SQL ASSIGNMENT

DIKSHA

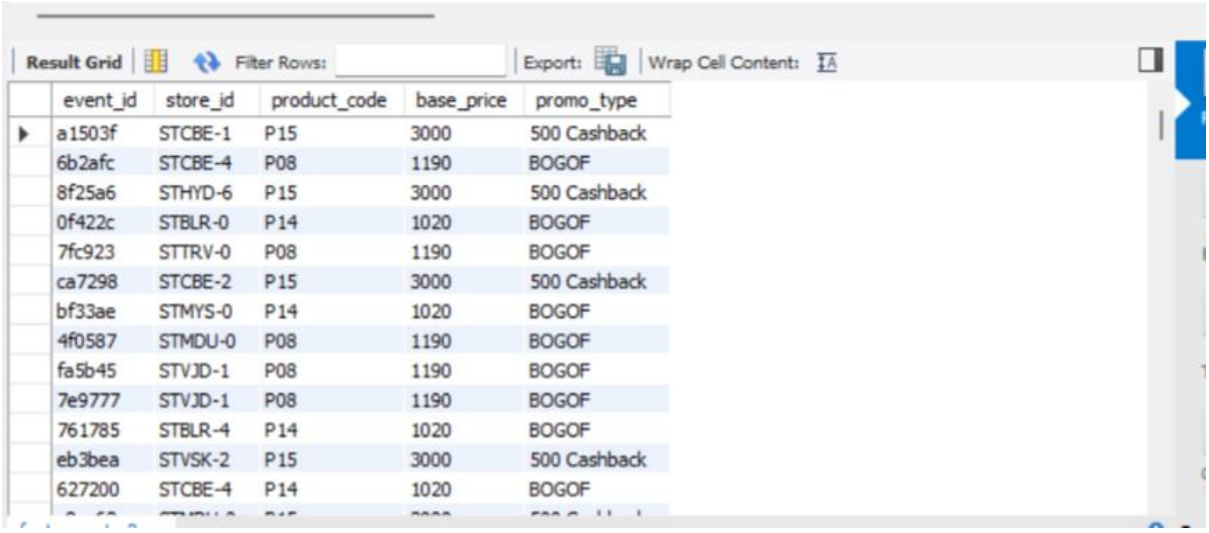
26271009

Q1. Basic Filtering – High-Value Products. Find all event records where the base_price is greater than 1,000.

SOL – 1:

```
SELECT
   event_id,
   store_id,
   product_code,
   base_price,
   promo_type
FROM fact_events
WHERE base_price > 1000;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

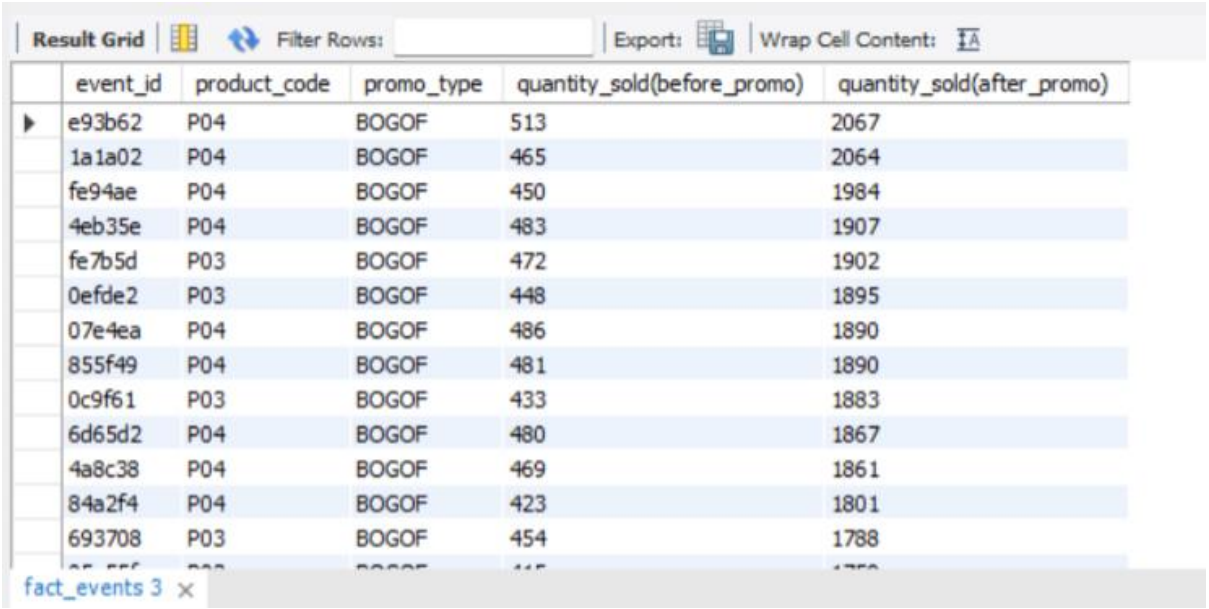
event_id	store_id	product_code	base_price	promo_type
a1503f	STCBE-1	P15	3000	500 Cashback
6b2afc	STCBE-4	P08	1190	BOGOF
8f25a6	STHYD-6	P15	3000	500 Cashback
0f422c	STBLR-0	P14	1020	BOGOF
7fc923	STTRV-0	P08	1190	BOGOF
ca7298	STCBE-2	P15	3000	500 Cashback
bf33ae	STMYS-0	P14	1020	BOGOF
4f0587	STMDU-0	P08	1190	BOGOF
fa5b45	STVJD-1	P08	1190	BOGOF
7e9777	STVJD-1	P08	1190	BOGOF
761785	STBLR-4	P14	1020	BOGOF
eb3bea	STVSK-2	P15	3000	500 Cashback
627200	STCBE-4	P14	1020	BOGOF

Q2. Sorting Promotional Events. Display all events where quantity sold after the promotion was greater than 100.

SOL-2:

```
SELECT
   event_id,
   product_code,
   promo_type,
   `quantity_sold(before_promo)`,
   `quantity_sold(after_promo)`
FROM fact_events
WHERE `quantity_sold(after_promo)` > 100
ORDER BY `quantity_sold(after_promo)` DESC;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

event_id	product_code	promo_type	quantity_sold(before_promo)	quantity_sold(after_promo)
e93b62	P04	BOGOF	513	2067
1a1a02	P04	BOGOF	465	2064
fe94ae	P04	BOGOF	450	1984
4eb35e	P04	BOGOF	483	1907
fe7b5d	P03	BOGOF	472	1902
0efde2	P03	BOGOF	448	1895
07e4ea	P04	BOGOF	486	1890
855f49	P04	BOGOF	481	1890
0c9f61	P03	BOGOF	433	1883
6d65d2	P04	BOGOF	480	1867
4a8c38	P04	BOGOF	469	1861
84a2f4	P04	BOGOF	423	1801
693708	P03	BOGOF	454	1788

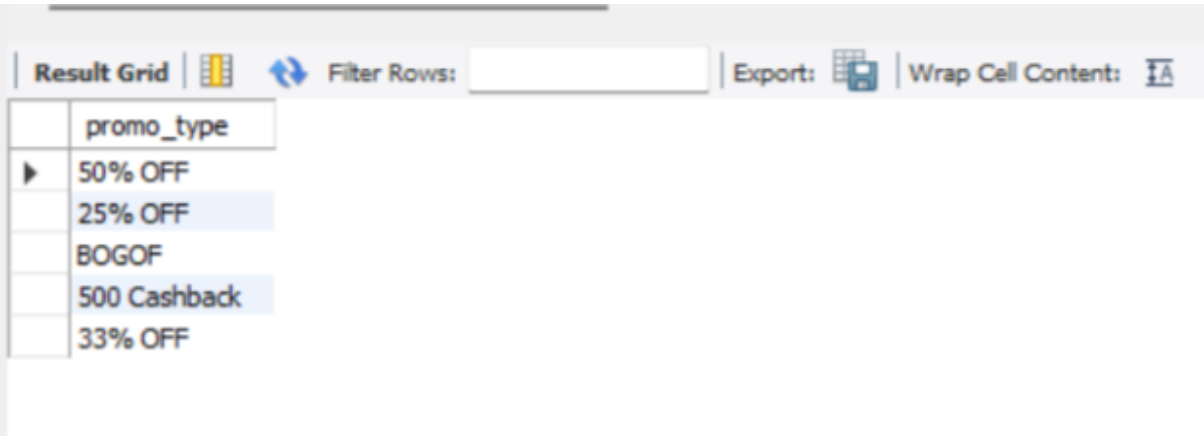
fact_events 3

Q3. DISTINCT Promotion Types. Find all unique promotion types used in the dataset.

SOL-3

```
SELECT DISTINCT promo_type
FROM fact_events;
```

OUTPUT:



promo_type
50% OFF
25% OFF
BOGOF
500 Cashback
33% OFF

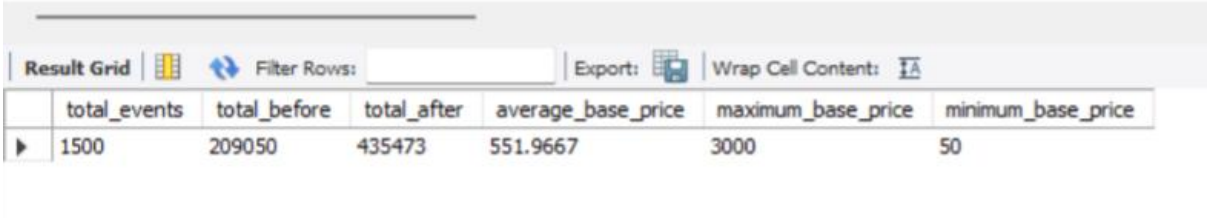
Q4. Basic Aggregation. Calculate the following for the complete fact_events table:

SOL-4

```
SELECT
    COUNT(*) AS total_events,
    SUM(`quantity_sold(before_promo)`) AS total_before,
    SUM(`quantity_sold(after_promo)`) AS total_after,
    AVG(base_price) AS average_base_price,
    MAX(base_price) AS maximum_base_price,
    MIN(base_price) AS minimum_base_price
```

FROM fact_events;

OUTPUT:



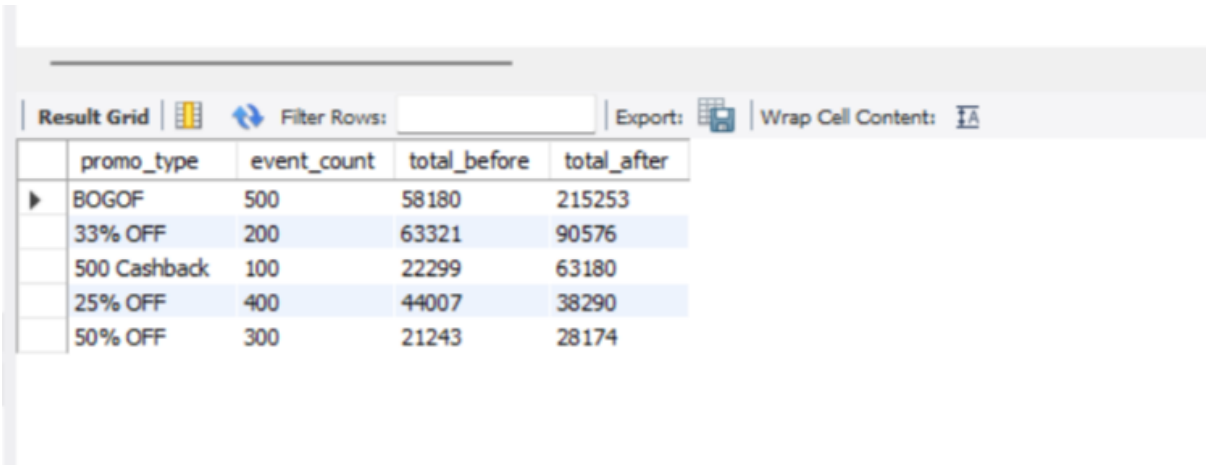
total_events	total_before	total_after	average_base_price	maximum_base_price	minimum_base_price
1500	209050	435473	551.9667	3000	50

Q5. Sales Volume by Promotion Type. For each promo_type, calculate:

SOL-5:

```
SELECT
   promo_type,
   COUNT(*) AS event_count,
   SUM(`quantity_sold(before_promo)`) AS total_before,
   SUM(`quantity_sold(after_promo)`) AS total_after
FROM fact_events
GROUP BY promo_type
ORDER BY total_after DESC;
```

OUTPUT:



promo_type	event_count	total_before	total_after
BOGOF	500	58180	215253
33% OFF	200	63321	90576
500 Cashback	100	22299	63180
25% OFF	400	44007	38290
50% OFF	300	21243	28174

Q6. Promotion Uplift. For every promotion type, calculate:

- Total quantity before promotion

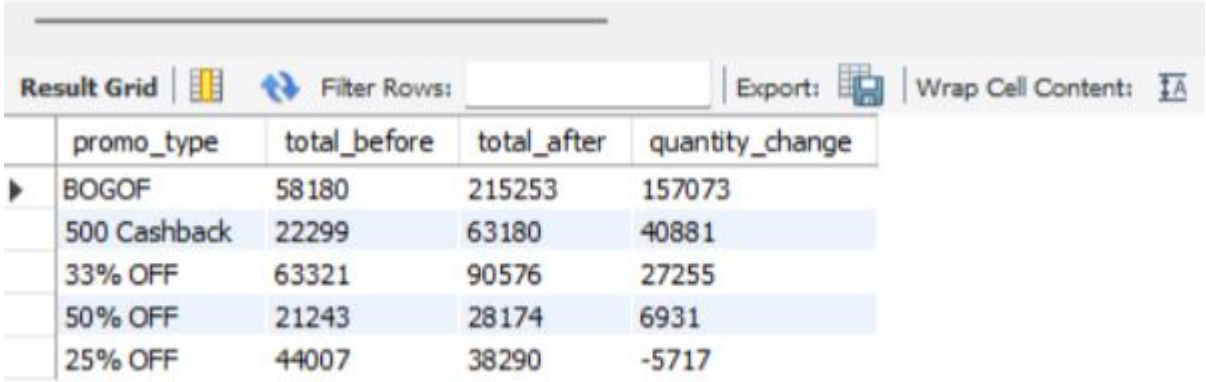
- Total quantity after promotion

- Quantity increase/decrease

SOL-6.

```
SELECT
   promo_type,
   SUM(`quantity_sold(before_promo)`) AS total_before,
   SUM(`quantity_sold(after_promo)`) AS total_after,
   SUM(`quantity_sold(after_promo)`)
      - SUM(`quantity_sold(before_promo)`) AS quantity_change
FROM fact_events
GROUP BY promo_type
ORDER BY quantity_change DESC;
```

OUTPUT:



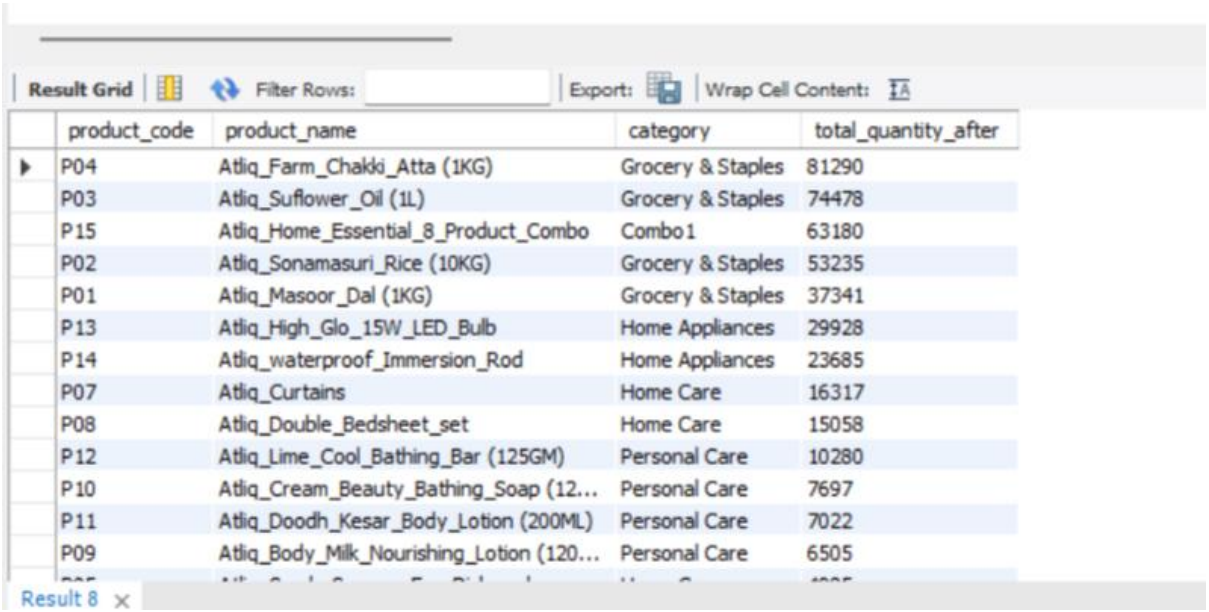
promo_type	total_before	total_after	quantity_change
BOGOF	58180	215253	157073
500 Cashback	22299	63180	40881
33% OFF	63321	90576	27255
50% OFF	21243	28174	6931
25% OFF	44007	38290	-5717

Q7. Product Performance. Using fact_events and dim_products, calculate total quantity sold after promotion for every product.

SOL-7.

```
SELECT
    p.product_code,
    p.product_name,
    p.category,
    SUM(f.`quantity_sold(after_promo)`) AS total_quantity_after
FROM fact_events f
INNER JOIN dim_products p
    ON f.product_code = p.product_code
GROUP BY
    p.product_code,
    p.product_name,
    p.category
ORDER BY total_quantity_after DESC;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_code	product_name	category	total_quantity_after
P04	Atliq_Farm_Chakki_Atta (1KG)	Grocery & Staples	81290
P03	Atliq_Suflower_Oil (1L)	Grocery & Staples	74478
P15	Atliq_Home_Essential_8_Product_Combo	Combo1	63180
P02	Atliq_Sonamasuri_Rice (10KG)	Grocery & Staples	53235
P01	Atliq_Masoor_Dal (1KG)	Grocery & Staples	37341
P13	Atliq_High_Glo_15W_LED_Bulb	Home Appliances	29928
P14	Atliq_waterproof_Immersion_Rod	Home Appliances	23685
P07	Atliq_Curtains	Home Care	16317
P08	Atliq_Double_Bedsheet_set	Home Care	15058
P12	Atliq_Lime_Cool_Bathing_Bar (125GM)	Personal Care	10280
P10	Atliq_Cream_Beauty_Bathing_Soap (12...	Personal Care	7697
P11	Atliq_Doodh_Kesar_Body_Lotion (200ML)	Personal Care	7022
P09	Atliq_Body_Milk_Nourishing_Lotion (120...	Personal Care	6505

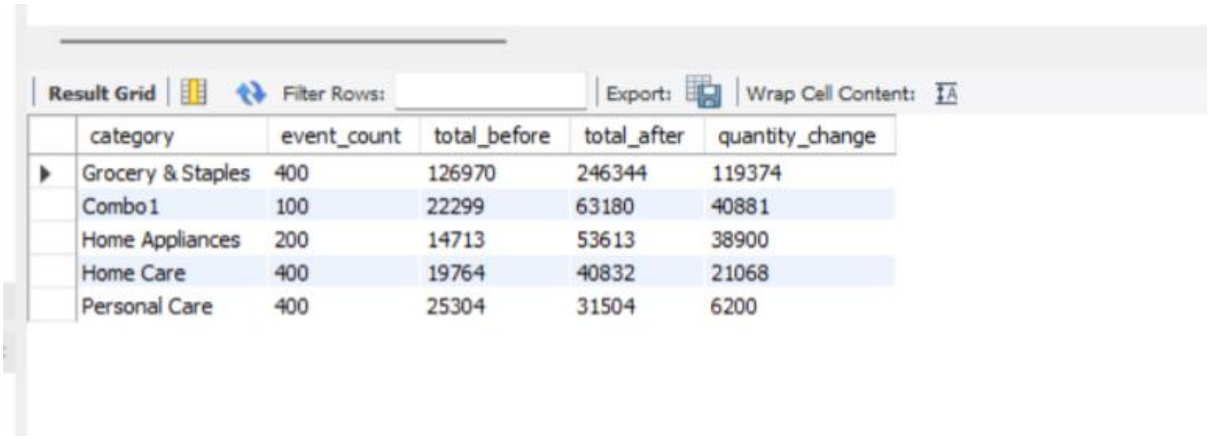
Result 8

Q8. Category-Level Performance. Using fact_events and dim_products, calculate for every product category:

SOL-8.

```
SELECT
    p.category,
    COUNT(*) AS event_count,
    SUM(f.`quantity_sold(before_promo)`) AS total_before,
    SUM(f.`quantity_sold(after_promo)`) AS total_after,
    SUM(f.`quantity_sold(after_promo)`)
        - SUM(f.`quantity_sold(before_promo)`) AS quantity_change
FROM fact_events f
INNER JOIN dim_products p
    ON f.product_code = p.product_code
GROUP BY p.category
ORDER BY total_after DESC;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	event_count	total_before	total_after	quantity_change
Grocery & Staples	400	126970	246344	119374
Combo1	100	22299	63180	40881
Home Appliances	200	14713	53613	38900
Home Care	400	19764	40832	21068
Personal Care	400	25304	31504	6200

Q9. Store Performance

Using fact_events and dim_stores, calculate for every city:

- Number of promotional events

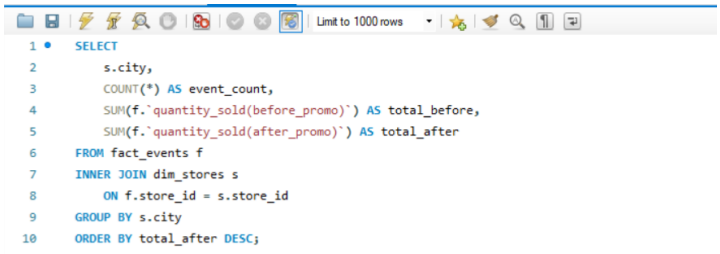
- Total quantity before promotion

- Total quantity after promotion

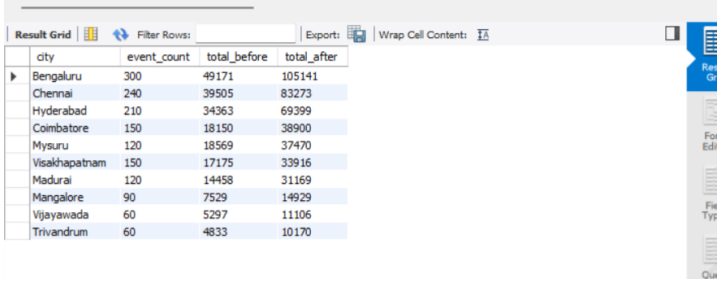
SOL-9

```
SELECT
    s.city,
    COUNT(*) AS event_count,
    SUM(f.`quantity_sold(before_promo)`) AS total_before,
    SUM(f.`quantity_sold(after_promo)`) AS total_after
FROM fact_events f
INNER JOIN dim_stores s
    ON f.store_id = s.store_id
GROUP BY s.city
ORDER BY total_after DESC;
```

OUTPUT:



```
SELECT
    s.city,
    COUNT(*) AS event_count,
    SUM(f.`quantity_sold(before_promo)`) AS total_before,
    SUM(f.`quantity_sold(after_promo)`) AS total_after
FROM fact_events f
INNER JOIN dim_stores s
    ON f.store_id = s.store_id
GROUP BY s.city
ORDER BY total_after DESC;
```



city	event_count	total_before	total_after
Bengaluru	300	49171	105141
Chennai	240	39505	83273
Hyderabad	210	34363	69399
Coimbatore	150	18150	38900
Mysuru	120	18569	37470
Visakhapatnam	150	17175	33916
Madurai	120	14458	31169
Mangalore	90	7529	14929
Vijayawada	60	5297	11106
Trivandrum	60	4833	10170

Q10. Campaign Performance

Using fact_events and dim_campaigns, calculate for each campaign:

- Campaign name

- Start date

- End date

- Number of events

- Total quantity before promotion

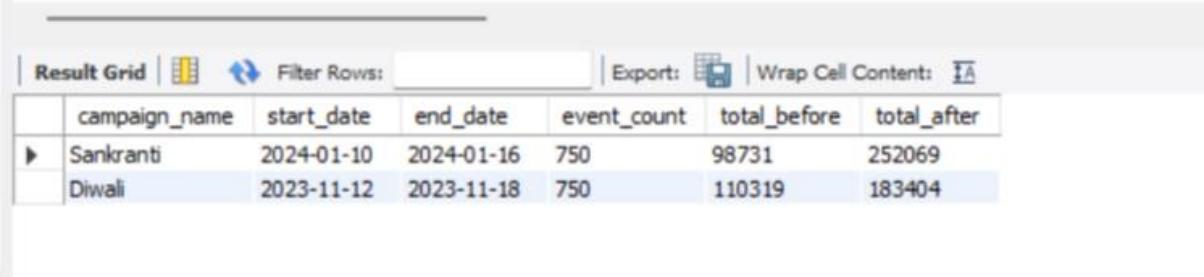
- Total quantity after promotion

SOL-10

```
SELECT
   c.campaign_name,
   c.start_date,
   c.end_date,
   COUNT(*) AS event_count,
   SUM(f.`quantity_sold(before_promo)`) AS total_before,
   SUM(f.`quantity_sold(after_promo)`) AS total_after
FROM fact_events f
INNER JOIN dim_campaigns c
   ON f.campaign_id = c.campaign_id
GROUP BY
   c.campaign_id,
   c.campaign_name,
   c.start_date,
   c.end_date
```

ORDER BY total_after DESC;

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

campaign_name	start_date	end_date	event_count	total_before	total_after
Sankranti	2024-01-10	2024-01-16	750	98731	252069
Diwali	2023-11-12	2023-11-18	750	110319	183404

Q11. Product Category with HAVING

Find product categories where the total quantity sold after promotion is greater than 1,000.

SOL-11

```
SELECT
    p.category,
    SUM(f.`quantity_sold(after_promo)`) AS total_quantity_after,
    AVG(f.base_price) AS average_base_price
FROM fact_events f
INNER JOIN dim_products p
    ON f.product_code = p.product_code
GROUP BY p.category
HAVING SUM(f.`quantity_sold(after_promo)`) > 1000
ORDER BY total_quantity_after DESC;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	total_quantity_after	average_base_price
Grocery & Staples	246344	385.0000
Combo1	63180	3000.0000
Home Appliances	53613	685.0000
Home Care	40832	490.0000
Personal Care	31504	102.3750

Q12. Store + Category Analysis

Using fact_events, dim_stores and dim_products, calculate total quantity sold after promotion for every combination of:

- City

- Product category

Display:

- city

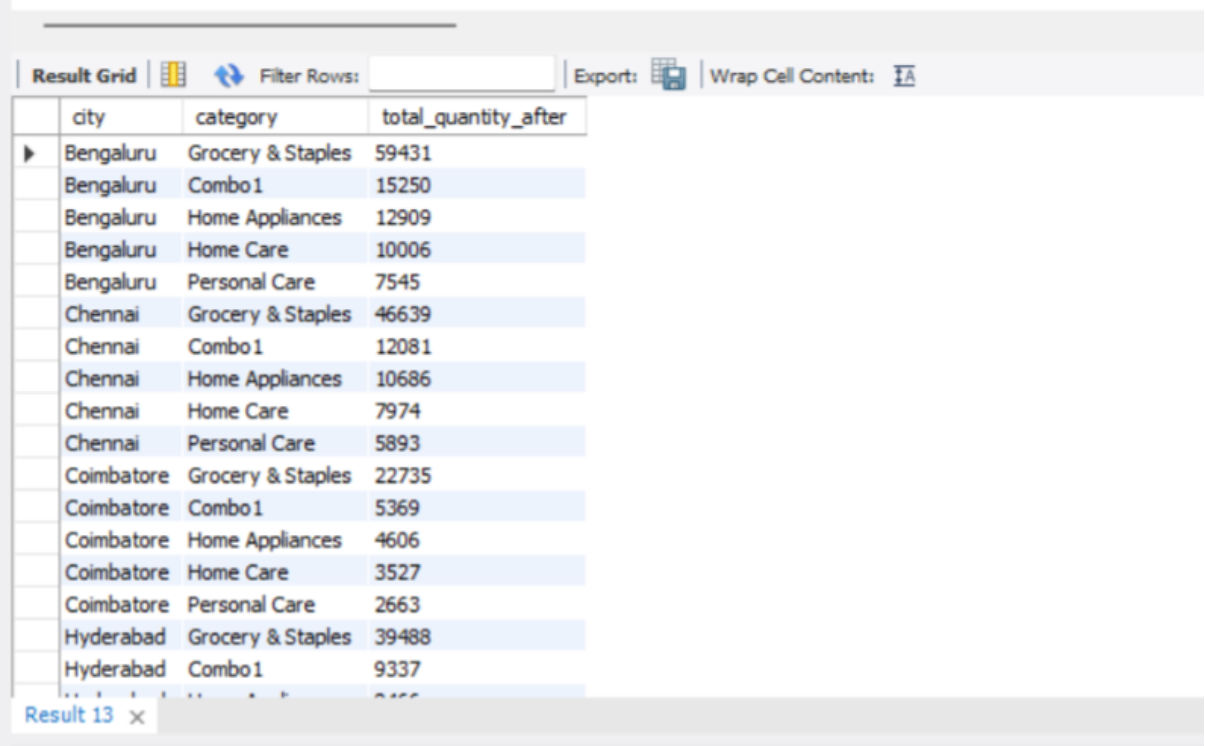
- category

- total quantity after promotion

SOL-12:

```
SELECT
   s.city,
   p.category,
   SUM(f.`quantity_sold(after_promo)`) AS total_quantity_after
FROM fact_events f
INNER JOIN dim_stores s
   ON f.store_id = s.store_id
INNER JOIN dim_products p
   ON f.product_code = p.product_code
GROUP BY
   s.city,
   p.category
ORDER BY
   s.city,
   total_quantity_after DESC;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

city	category	total_quantity_after
Bengaluru	Grocery & Staples	59431
Bengaluru	Combo1	15250
Bengaluru	Home Appliances	12909
Bengaluru	Home Care	10006
Bengaluru	Personal Care	7545
Chennai	Grocery & Staples	46639
Chennai	Combo1	12081
Chennai	Home Appliances	10686
Chennai	Home Care	7974
Chennai	Personal Care	5893
Coimbatore	Grocery & Staples	22735
Coimbatore	Combo1	5369
Coimbatore	Home Appliances	4606
Coimbatore	Home Care	3527
Coimbatore	Personal Care	2663
Hyderabad	Grocery & Staples	39488
Hyderabad	Combo1	9337

Result 13

Q13. Promotion Effectiveness by Product

For each product, calculate:

- Product name

- Category

- Total quantity before promotion

- Total quantity after promotion

- Quantity change

- Percentage change

SOL-13

```
SELECT
    p.product_name,
```

```
    p.category,
    SUM(f.`quantity_sold(before_promo)`) AS total_before,
    SUM(f.`quantity_sold(after_promo)`) AS total_after,

    SUM(f.`quantity_sold(after_promo)`)
      - SUM(f.`quantity_sold(before_promo)`) AS quantity_change,

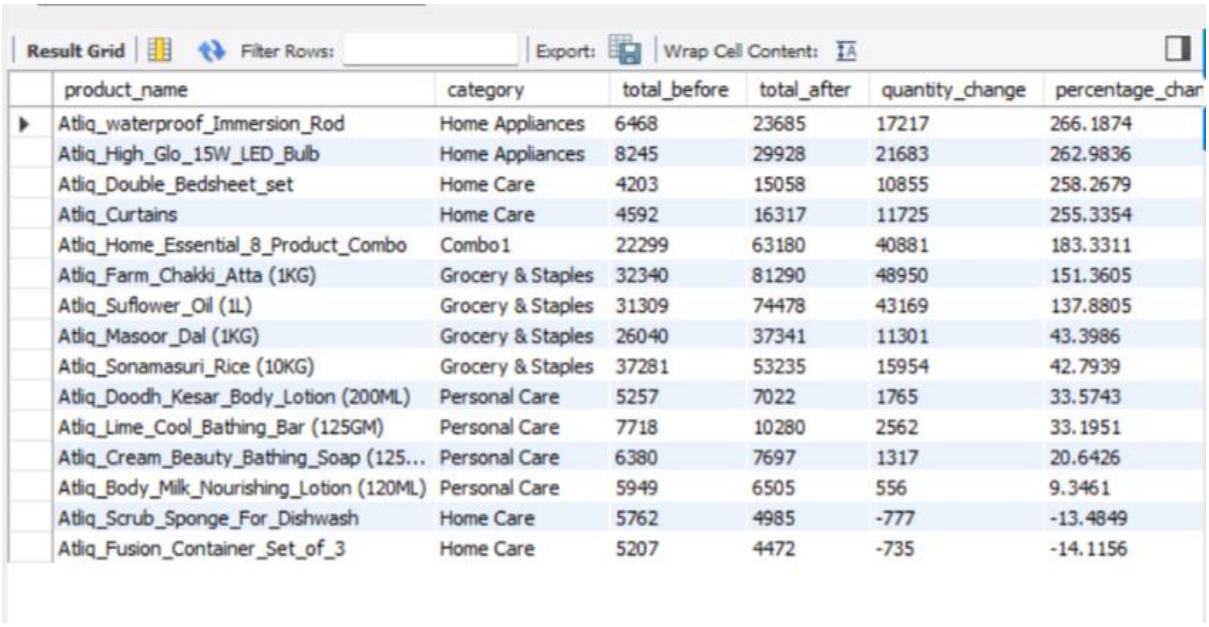
    (
      (
        SUM(f.`quantity_sold(after_promo)`)
        - SUM(f.`quantity_sold(before_promo)`)
      )
      / NULLIF(SUM(f.`quantity_sold(before_promo)`), 0)
    ) * 100 AS percentage_change

FROM fact_events f
INNER JOIN dim_products p
    ON f.product_code = p.product_code

GROUP BY
    p.product_code,
    p.product_name,
    p.category

ORDER BY percentage_change DESC;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	category	total_before	total_after	quantity_change	percentage_char
Atliq_waterproof_Immersion_Rod	Home Appliances	6468	23685	17217	266.1874
Atliq_High_Glo_15W_LED_Bulb	Home Appliances	8245	29928	21683	262.9836
Atliq_Double_Bedsheet_set	Home Care	4203	15058	10855	258.2679
Atliq_Curtains	Home Care	4592	16317	11725	255.3354
Atliq_Home_Essential_8_Product_Combo	Combo1	22299	63180	40881	183.3311
Atliq_Farm_Chakki_Atta (1KG)	Grocery & Staples	32340	81290	48950	151.3605
Atliq_Suflower_Oil (1L)	Grocery & Staples	31309	74478	43169	137.8805
Atliq_Masoor_Dal (1KG)	Grocery & Staples	26040	37341	11301	43.3986
Atliq_Sonamasuri_Rice (10KG)	Grocery & Staples	37281	53235	15954	42.7939
Atliq_Doodh_Kesar_Body_Lotion (200ML)	Personal Care	5257	7022	1765	33.5743
Atliq_Lime_Cool_Bathing_Bar (125GM)	Personal Care	7718	10280	2562	33.1951
Atliq_Cream_Beauty_Bathing_Soap (125...	Personal Care	6380	7697	1317	20.6426
Atliq_Body_Milk_Nourishing_Lotion (120ML)	Personal Care	5949	6505	556	9.3461
Atliq_Scrub_Sponge_For_Dishwash	Home Care	5762	4985	-777	-13.4849
Atliq_Fusion_Container_Set_of_3	Home Care	5207	4472	-735	-14.1156

Q14. Campaign and Promotion Type Analysis. For each campaign and promo_type combination, calculate:

- Number of events

- Total quantity before promotion

- Total quantity after promotion

- Quantity change

SOL-14

```
SELECT
    c.campaign_name,
    f.promo_type,

    COUNT(*) AS event_count,

    SUM(f.`quantity_sold(before_promo)`) AS total_before,

    SUM(f.`quantity_sold(after_promo)`) AS total_after,
```

```
    SUM(f.`quantity_sold(after_promo)`)
        - SUM(f.`quantity_sold(before_promo)`) AS quantity_change

FROM fact_events f

INNER JOIN dim_campaigns c
    ON f.campaign_id = c.campaign_id

GROUP BY
    c.campaign_id,
    c.campaign_name,
    f.promo_type

ORDER BY
    c.campaign_name,
    quantity_change DESC;
```

OUTPUT:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

campaign_name	promo_type	event_count	total_before	total_after	quantity_change
Diwali	500 Cashback	50	16791	50769	33978
Diwali	BOGOF	200	10024	34461	24437
Diwali	33% OFF	100	29152	43117	13965
Diwali	50% OFF	200	16843	22074	5231
Diwali	25% OFF	200	37509	32983	-4526
Sankranti	BOGOF	300	48156	180792	132636
Sankranti	33% OFF	100	34169	47459	13290
Sankranti	500 Cashback	50	5508	12411	6903
Sankranti	50% OFF	100	4400	6100	1700
Sankranti	25% OFF	200	6498	5307	-1191

Q15. Product Revenue Before and After Promotion

For each product, calculate:

1. Revenue before promotion =

base_price × quantity_sold(before_promo)

2. Revenue after promotion =

base_price × quantity_sold(after_promo)

3. Revenue difference =

Revenue after - Revenue before

SOL-15

```
SELECT
    p.product_name,
    p.category,

    SUM(
        f.base_price * f.`quantity_sold(before_promo)`
    ) AS revenue_before,

    SUM(
        f.base_price * f.`quantity_sold(after_promo)`
    ) AS revenue_after,

    SUM(
```

```
    f.base_price * f.`quantity_sold(after_promo)`
  )
  -
  SUM(
    f.base_price * f.`quantity_sold(before_promo)`
  ) AS revenue_difference

FROM fact_events f

INNER JOIN dim_products p
  ON f.product_code = p.product_code

GROUP BY
  p.product_code,
  p.product_name,
  p.category

ORDER BY revenue_difference DESC;
```

OUTPUT:

Q16. Classify Promotion Performance

For every promotion type, calculate total quantity before and after promotion.

Then classify the promotion using CASE:

- Percentage change >= 50% → "High Impact"

- Percentage change >= 20% → "Medium Impact"

- Percentage change < 20% → "Low Impact"

SOL-16

```
WITH promotion_summary AS (
   SELECT
      promo_type,

      SUM(`quantity_sold(before_promo)`) AS total_before,

      SUM(`quantity_sold(after_promo)`) AS total_after,

      (
         (
            SUM(`quantity_sold(after_promo)`)
            - SUM(`quantity_sold(before_promo)`)
         )
         / NULLIF(SUM(`quantity_sold(before_promo)`), 0)
      ) * 100 AS percentage_change

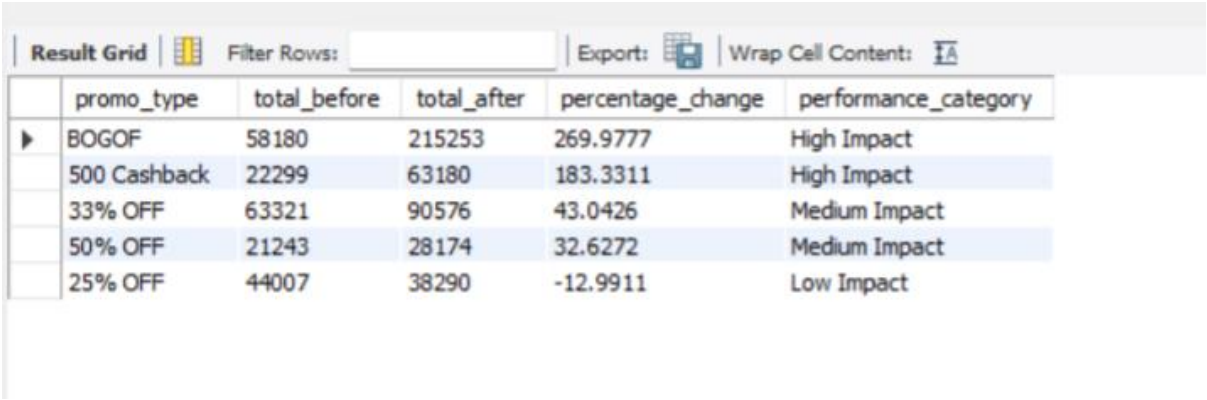
   FROM fact_events
   GROUP BY promo_type
)
```

```
SELECT
    promo_type,
    total_before,
    total_after,
    percentage_change,

    CASE
        WHEN percentage_change >= 50 THEN 'High Impact'
        WHEN percentage_change >= 20 THEN 'Medium Impact'
        ELSE 'Low Impact'
    END AS performance_category

FROM promotion_summary
ORDER BY percentage_change DESC;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

promo_type	total_before	total_after	percentage_change	performance_category
BOGOF	58180	215253	269.9777	High Impact
500 Cashback	22299	63180	183.3311	High Impact
33% OFF	63321	90576	43.0426	Medium Impact
50% OFF	21243	28174	32.6272	Medium Impact
25% OFF	44007	38290	-12.9911	Low Impact

Q17. Top Products Within Each Category

Using a CTE:

1. Calculate total quantity sold after promotion for every product.

2. Rank products within each category based on total quantity sold after promotion.

3. Return only the top 2 products from every category.

SOL-17

```
WITH product_sales AS (
   SELECT
      p.category,
      p.product_code,
      p.product_name,
      SUM(f.`quantity_sold(after_promo)`) AS total_quantity_after

   FROM fact_events f

   INNER JOIN dim_products p
      ON f.product_code = p.product_code

   GROUP BY
      p.category,
      p.product_code,
      p.product_name
),
```

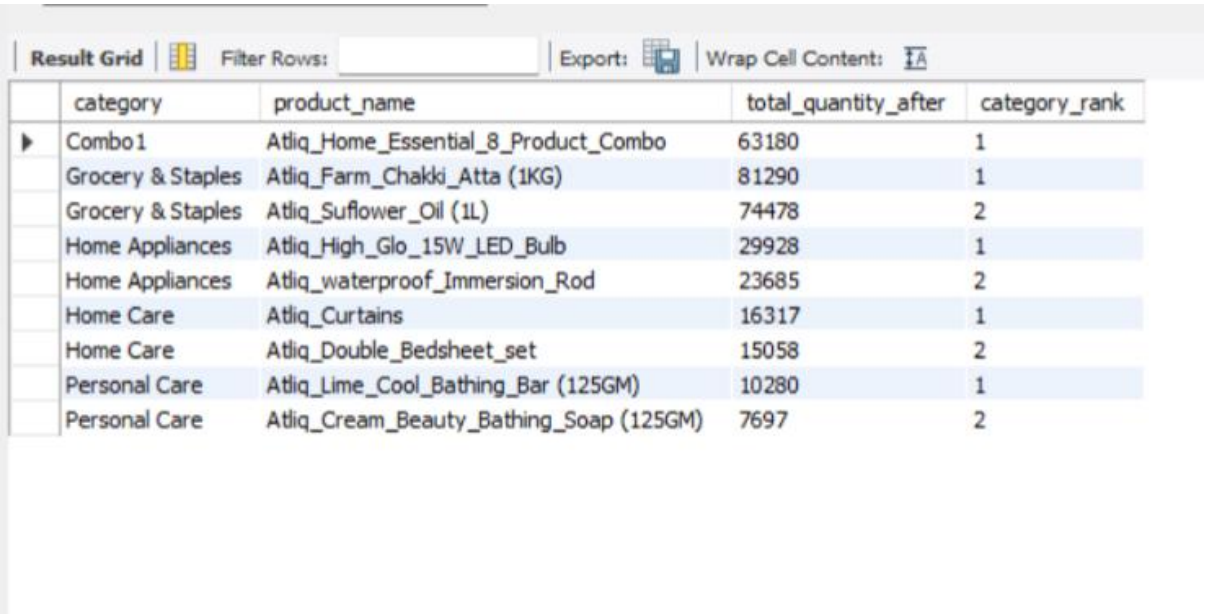
```
ranked_products AS (
    SELECT
        category,
        product_name,
        total_quantity_after,

        DENSE_RANK() OVER (
            PARTITION BY category
            ORDER BY total_quantity_after DESC
        ) AS category_rank

    FROM product_sales
)

SELECT
    category,
    product_name,
    total_quantity_after,
    category_rank
FROM ranked_products
WHERE category_rank <= 2
ORDER BY
    category,
    category_rank;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	product_name	total_quantity_after	category_rank
Combo1	Atliq_Home_Essential_8_Product_Combo	63180	1
Grocery & Staples	Atliq_Farm_Chakki_Atta (1KG)	81290	1
Grocery & Staples	Atliq_Suflower_Oil (1L)	74478	2
Home Appliances	Atliq_High_Glo_15W_LED_Bulb	29928	1
Home Appliances	Atliq_waterproof_Immersion_Rod	23685	2
Home Care	Atliq_Curtains	16317	1
Home Care	Atliq_Double_Bedsheet_set	15058	2
Personal Care	Atliq_Lime_Cool_Bathing_Bar (125GM)	10280	1
Personal Care	Atliq_Cream_Beauty_Bathing_Soap (125GM)	7697	2

Q18. Best-Performing Stores Within Each City. Calculate total quantity sold after promotion for each store.

SOL-18.

```
WITH store_sales AS (
  SELECT
    s.city,
    s.store_id,
    SUM(f.`quantity_sold(after_promo)`) AS total_quantity_after

  FROM fact_events f

  INNER JOIN dim_stores s
    ON f.store_id = s.store_id

  GROUP BY
    s.city,
```

```
        s.store_id
),

ranked_stores AS (
    SELECT
        city,
        store_id,
        total_quantity_after,

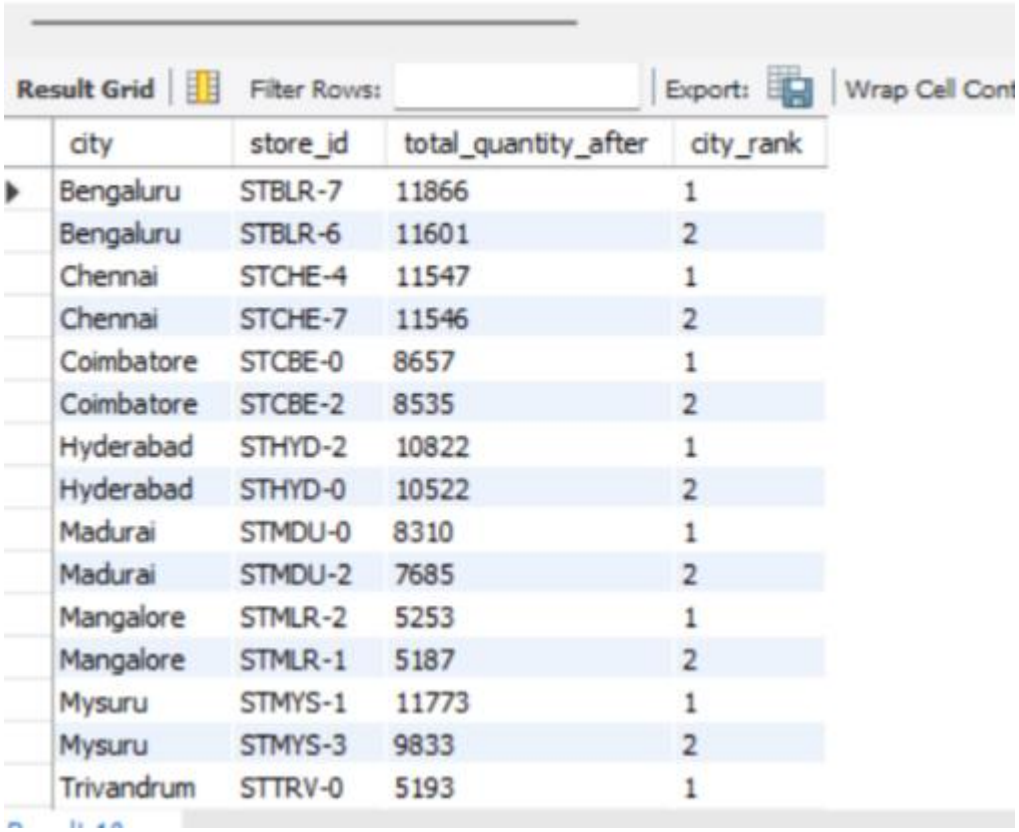
        DENSE_RANK() OVER (
            PARTITION BY city
            ORDER BY total_quantity_after DESC
        ) AS city_rank

    FROM store_sales
)

SELECT
    city,
    store_id,
    total_quantity_after,
    city_rank
FROM ranked_stores
WHERE city_rank <= 2
ORDER BY
    city,
```

city_rank;

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Cont

city	store_id	total_quantity_after	city_rank
Bengaluru	STBLR-7	11866	1
Bengaluru	STBLR-6	11601	2
Chennai	STCHE-4	11547	1
Chennai	STCHE-7	11546	2
Coimbatore	STCBE-0	8657	1
Coimbatore	STCBE-2	8535	2
Hyderabad	STHYD-2	10822	1
Hyderabad	STHYD-0	10522	2
Madurai	STMDU-0	8310	1
Madurai	STMDU-2	7685	2
Mangalore	STMLR-2	5253	1
Mangalore	STMLR-1	5187	2
Mysuru	STMYS-1	11773	1
Mysuru	STMYS-3	9833	2
Trivandrum	STTRV-0	5193	1

Q19. Campaign-Level Product Performance

For every campaign and product:

Calculate:

- Total quantity before promotion

- Total quantity after promotion

- Quantity change

- Percentage change

SOL-19

WITH campaign_product AS (

```
    SELECT
        c.campaign_id,
        c.campaign_name,
        p.product_code,
        p.product_name,

        SUM(f.`quantity_sold(before_promo)`) AS total_before,

        SUM(f.`quantity_sold(after_promo)`) AS total_after

    FROM fact_events f

    INNER JOIN dim_campaigns c
        ON f.campaign_id = c.campaign_id

    INNER JOIN dim_products p
        ON f.product_code = p.product_code

    GROUP BY
        c.campaign_id,
        c.campaign_name,
        p.product_code,
        p.product_name
),

performance AS (
```

```
    SELECT
        campaign_id,
        campaign_name,
        product_code,
        product_name,
        total_before,
        total_after,

        total_after - total_before AS quantity_change,

        (
            (total_after - total_before)
            / NULLIF(total_before, 0)
        ) * 100 AS percentage_change

    FROM campaign_product
),

ranked_products AS (
    SELECT
        campaign_name,
        product_name,
        total_before,
        total_after,
        quantity_change,
        percentage_change,
```

```
        DENSE_RANK() OVER (
            PARTITION BY campaign_id
            ORDER BY percentage_change DESC
        ) AS campaign_rank

    FROM performance
)

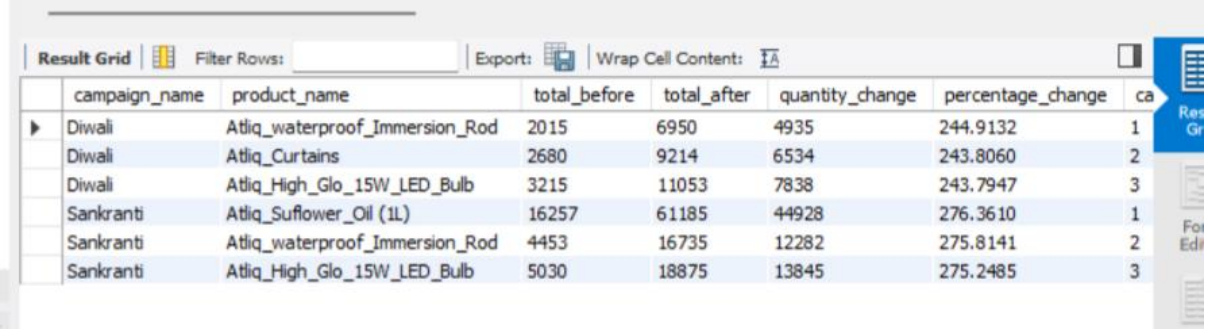
SELECT
    campaign_name,
    product_name,
    total_before,
    total_after,
    quantity_change,
    percentage_change,
    campaign_rank

FROM ranked_products

WHERE campaign_rank <= 3

ORDER BY
    campaign_name,
    campaign_rank;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

campaign_name	product_name	total_before	total_after	quantity_change	percentage_change	ca
Diwali	Atliq_waterproof_Immersion_Rod	2015	6950	4935	244.9132	1
Diwali	Atliq_Curtains	2680	9214	6534	243.8060	2
Diwali	Atliq_High_Glo_15W_LED_Bulb	3215	11053	7838	243.7947	3
Sankranti	Atliq_Suflower_Oil (1L)	16257	61185	44928	276.3610	1
Sankranti	Atliq_waterproof_Immersion_Rod	4453	16735	12282	275.8141	2
Sankranti	Atliq_High_Glo_15W_LED_Bulb	5030	18875	13845	275.2485	3

Q20. Complete Promotional Performance Analysis

Create a complete analytical report at the product-category level.

For every product, calculate:

- Product name

- Category

- Number of promotional events

- Total quantity before promotion

- Total quantity after promotion

- Quantity change

- Percentage change

- Revenue before promotion

- Revenue after promotion

- Revenue change

- Average base price

- Product rank within its category

SOL-20.

```
WITH product_summary AS (

    SELECT
        p.product_code,
        p.product_name,
        p.category,

        COUNT(*) AS event_count,

        SUM(f.`quantity_sold(before_promo)`) AS total_before,

        SUM(f.`quantity_sold(after_promo)`) AS total_after,

        SUM(
            f.base_price * f.`quantity_sold(before_promo)`
        ) AS revenue_before,

        SUM(
            f.base_price * f.`quantity_sold(after_promo)`
        ) AS revenue_after,

        AVG(f.base_price) AS average_base_price

    FROM fact_events f

    INNER JOIN dim_products p
```

```
        ON f.product_code = p.product_code

    GROUP BY
        p.product_code,
        p.product_name,
        p.category
),

performance AS (

    SELECT
        product_code,
        product_name,
        category,
        event_count,
        total_before,
        total_after,

        total_after - total_before AS quantity_change,

        (
            (total_after - total_before)
            / NULLIF(total_before, 0)
        ) * 100 AS percentage_change,

        revenue_before,
```

```
        revenue_after,

        revenue_after - revenue_before AS revenue_change,

        average_base_price

    FROM product_summary
),

ranked_products AS (

    SELECT
        product_name,
        category,
        event_count,
        total_before,
        total_after,
        quantity_change,
        percentage_change,
        revenue_before,
        revenue_after,
        revenue_change,
        average_base_price,

        DENSE_RANK() OVER (
            PARTITION BY category
```

```
            ORDER BY revenue_change DESC
        ) AS product_rank

    FROM performance
)

SELECT
    product_name,
    category,
    event_count,
    total_before,
    total_after,
    quantity_change,
    percentage_change,
    revenue_before,
    revenue_after,
    revenue_change,
    average_base_price,
    product_rank

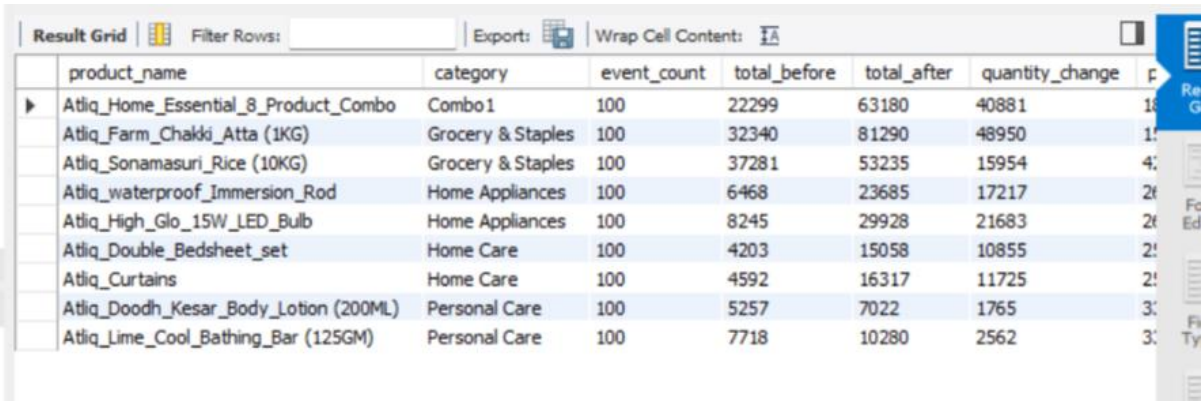
FROM ranked_products

WHERE product_rank <= 2

ORDER BY
    category,
```

product_rank;

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	category	event_count	total_before	total_after	quantity_change	p
Atliq_Home_Essential_8_Product_Combo	Combo1	100	22299	63180	40881	1
Atliq_Farm_Chakki_Atta (1KG)	Grocery & Staples	100	32340	81290	48950	1
Atliq_Sonamasuri_Rice (10KG)	Grocery & Staples	100	37281	53235	15954	4
Atliq_waterproof_Immersion_Rod	Home Appliances	100	6468	23685	17217	2
Atliq_High_Glo_15W_LED_Bulb	Home Appliances	100	8245	29928	21683	2
Atliq_Double_Bedsheet_set	Home Care	100	4203	15058	10855	2
Atliq_Curtains	Home Care	100	4592	16317	11725	2
Atliq_Doodh_Kesar_Body_Lotion (200ML)	Personal Care	100	5257	7022	1765	3
Atliq_Lime_Cool_Bathing_Bar (125GM)	Personal Care	100	7718	10280	2562	3